macOS · zsh

APPLE SILICON
AND INTEL

Every line is a command with an answer you can read. Tick in pencil; the numbers on the right are the chapters that fix a failure.

MACHINE

☐	`uname -m`	prints `arm64` or `x86_64` — and you know which you have	02
☐	`xcode-select -p`	command line tools installed	02
☐	`df -h /`	at least 50 GB free	26

SHELL AND PATH

☐	`echo $SHELL`	zsh, and you know `~/.zshrc` is the file you edit	03
☐	`echo $PATH | tr ':' '\n'`	`~/.local/bin` appears, and nothing is duplicated	04
☐	`which -a python3`	you can name every interpreter it lists	05

PYTHON

☐	`python3.11 -V`	a 3.11.x version, and it is not Apple's	05
☐	`python3.11 -m venv .venv`	creates the folder without error	06
☐	`source .venv/bin/activate`	`which python` then ends in `.venv/bin/python`	06
☐	`python -m pip -V`	the pip inside the environment, not a global one	07
☐	`uv --version`	installed, or you have deliberately chosen another tool	09, 11

GIT AND GITHUB

☐	`git -v` · `git config user.email`	both answer	15
☐	`ssh -T git@github.com`	greets you by username	17
☐	`cat .gitignore`	contains `.venv`, `.env`, `__pycache__`	15

EDITOR

☐	`code --version`	the `code` command works from any directory	14
☐	Status bar interpreter	shows the `.venv` interpreter for this project	14
☐	Breakpoint + F5	the debugger stops and shows your variables	14

ACCELERATOR

☐	`torch.backends.mps.is_available()`	True on Apple Silicon; False on Intel is correct	22
☐	Device selection in code	your script picks cuda / mps / cpu by itself	22

SECRETS AND CACHES

☐	`ls -l .env`	exists, mode `600`, and git ignores it	27
☐	Key loads from `.env`	and the tool fails with a clear message when it is missing	27
☐	`echo $HF_HOME`	you know where model weights are being written	26
☐	Provider spend limit	set, and you know the number	28

Ubuntu / Debian · bash

WORKSTATION OR SERVER

Where a line needs `sudo` and you do not have it, the manual's no-admin column applies — note it and move on.

MACHINE

☐	`lsb_release -a`	distribution and version noted	02
☐	`free -h` · `df -h /`	RAM known; at least 50 GB free	02, 26
☐	`gcc --version`	a compiler exists, or you accept that wheels must be prebuilt	12

SHELL AND PATH

☐	`echo $0`	bash, and you edit `~/.bashrc`	03
☐	`echo $PATH | tr ':' '\n'`	`~/.local/bin` present, once	04

PYTHON

☐	`python3.11 -V`	3.11.x available	05
☐	`python3.11 -m venv .venv`	succeeds — needs `python3.11-venv` on Debian family	05, 06
☐	`which python` after activating	points inside `.venv`	06
☐	`python -m pip check`	no broken dependencies	07, 12
☐	Lockfile rebuild	delete `.venv`, rebuild in one command, tests pass	13

GIT, GITHUB, SHELL TOOLS

☐	`git -v` · identity configured	both answer	15
☐	`ssh -T git@github.com`	greets you; key registered for this machine	17
☐	`tmux -V`	installed, and you have detached and reattached once	18
☐	`make --version`	present, and `make setup` works in your project	19
☐	`pre-commit --version`	hooks installed in this clone	19

GPU

☐	`nvidia-smi`	driver version and VRAM noted — or "no GPU" accepted	22
☐	`torch.cuda.is_available()`	agrees with the line above	22
☐	`torch.version.cuda`	not `None` if you expect GPU — otherwise you have the CPU wheel	22

DOCKER

☐	`docker run hello-world`	works without `sudo`	24
☐	`docker run --gpus all … nvidia-smi`	works, if you have a GPU	25
☐	`docker system df`	you have looked at it once and know what it means	26

SECRETS

Windows + WSL2 · bash

RECOMMENDED
ON WINDOWS

Two machines, one laptop. The rule that prevents most confusion: the driver and Docker Desktop live on Windows; your code, Python and git live inside Linux.

WSL2 ITSELF

☐	`wsl -l -v` (in PowerShell)	your distribution is listed at version 2	02
☐	`pwd` inside your project	a path under `/home/`, never under `/mnt/c/`	02
☐	`df -h /` and Windows free space	both have room; WSL2's disk grows on demand	26

PYTHON INSIDE LINUX

☐	`python3.11 -V`	3.11.x, installed inside WSL2 — not the Windows one	05
☐	`python3.11 -m venv .venv`	succeeds; `python3.11-venv` installed if needed	05, 06
☐	`which python` after activating	inside `.venv`, and not a `/mnt/c/` path	04, 06
☐	Lockfile rebuild	one command from clean, tests pass	13

GIT

☐	`git -v`, identity, `core.autocrlf input`	all three set inside WSL2	15
☐	`ssh -T git@github.com`	from inside WSL2, with a key generated there	17
☐	`git status` on a fresh clone	does not show every file as modified	15

EDITOR

☐	`code .` from a WSL2 shell	opens VS Code attached to WSL2 (green indicator, bottom left)	14
☐	Extensions	Python and Jupyter installed *in the WSL2 context*	14
☐	Interpreter	the `.venv` inside Linux, not a Windows path	14

GPU

☐	Driver installed on **Windows**	and nothing NVIDIA installed inside Linux	22
☐	`nvidia-smi` inside WSL2	reports the card	22
☐	`torch.cuda.is_available()`	True, with a `cu###` wheel installed in WSL2	22

DOCKER

☐	Docker Desktop → WSL2 integration	enabled for your distribution	24
☐	`docker run hello-world` in WSL2	works	24

SECRETS

☐	`ls -l .env`	mode `600`, git-ignored, loads	27
☐	No keys on the Windows side	one copy, inside the project, inside Linux	27

Name _____ Date _____

Windows native · PowerShell

NO ADMIN RIGHTS NEEDED

The path for locked-down machines. Everything here installs for your user only. A minority of tutorials will need translating; the manual's PowerShell lines do that for you.

SHELL

☐	`$PSVersionTable`	PowerShell 7.x preferred; 5.1 workable	03
☐	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`	set — otherwise activation scripts are blocked	06
☐	`$PROFILE`	you know the file and can edit it	04

PYTHON

☐	App execution aliases off	typing `python` does not open the Microsoft Store	05
☐	`py --list`	3.11 present	05
☐	`py -3.11 -m venv .venv`	creates the environment	06
☐	`.\.venv\Scripts\Activate.ps1`	activates without a policy error	06
☐	`where.exe python`	first result is inside `.venv\Scripts`	04
☐	`python -m pip -V`	the environment's pip	07
☐	Lockfile rebuild	delete `.venv`, rebuild in one command, tests pass	13

GIT

☐	`git -v`, identity, `core.autocrlf input`	all set	15
☐	`ssh -T git@github.com`	greets you; `ssh-agent` service running	17
☐	Line endings	a fresh clone does not show every file as changed	15

EDITOR AND TASKS

☐	Interpreter selected	`.venv\Scripts\python.exe` in the status bar	14
☐	Task runner	`just`, `nox` or scripts — `make` is not available here	19
☐	`pre-commit --version`	hooks installed	19

GPU

☐	`nvidia-smi`	driver noted, or "no GPU" accepted	22
☐	`torch.version.cuda`	matches your expectation, CPU or CUDA	22

SECRETS AND LIMITS

☐	`.env` git-ignored	and it loads through dotenv	27
☐	Key never printed	not in a cell, not in a log, not in a screenshot	27
☐	Provider spend limit	set, and you know the number	28
☐	Free disk	at least 50 GB, and you know where caches live	26

Name _____ Date _____

Cloud only · Colab and rented GPUs

BROWSER IS YOUR MACHINE

Every machine here is temporary. The checklist is therefore mostly about what survives the machine: your repository, your secrets store, your saved results and your bill.

BEFORE YOU START

☐	GitHub account and repository	your code has a home that is not the runtime	16
☐	Lockfile in the repository	one cell rebuilds the environment	13
☐	Billing alert	set at a number that would annoy you	23
☐	Provider spend limit	set on every API key	27, 28

COLAB SESSION

☐	GPU runtime selected	Runtime → Change runtime type	21
☐	`!nvidia-smi`	you know which card you were given, and its memory	21, 22
☐	`torch.cuda.is_available()`	True — or you have accepted a CPU session	22
☐	Secrets panel	keys stored there; no key in any cell	21, 27
☐	Drive mounted	you have saved a file and recovered it after a restart	21
☐	Clone-and-install cell	your repository runs unchanged in a fresh session	21
☐	`print(sys.executable)`	first cell of every notebook — you know your interpreter	20

RENTED MACHINES

☐	SSH key registered with the provider	and `~/.ssh/config` has a named host	17
☐	Sized by VRAM	the model fits, and you chose the smallest card it fits on	23
☐	tmux	long jobs run detached; a dropped connection is harmless	18
☐	Results written off the box	`rsync` or object storage, before the session ends	17, 23
☐	Checkpoints	an interrupted job resumes rather than restarts	21, 23
☐	Instance destroyed	dashboard shows nothing running, and no orphan volumes	23

NOTEBOOK DISCIPLINE

☐	Restart and Run All	the notebook works top to bottom in a fresh kernel	20
☐	Output stripped before committing	`nbstripout`, or paired files	19, 20
☐	Working code moved into a module	notebooks stay thin	20